Deep Learning-Based Walkable Area Segmentation and Traffic Obstacle Tracking for Pedestrians

Tianqi Li

March 17, 2025

Contents

Abstract

This study proposes a pedestrian area segmentation and multi-target traffic obstacle detection and tracking method combined with deep learning for pedestrian traffic scenes captured in real time by a monocular camera. All training is based on a self-designed first-person perspective dataset, which collects a large amount of pedestrian perspective data and related traffic status information. This study uses a multi-target detection model based on the YOLO11 series detector, and analyzes the accuracy and real-time performance of this model with the three trackers in detecting and tracking pedestrians and traffic participants (vehicles, bicycles, etc.) by comparing with tracking algorithms such as DeepSORT, ByteTrack and BoT-SORT. The experimental results show that this method achieves high-precision walkable area segmentation in a monocular video stream of a typical European city street, and can stably and continuously track and identify dynamic obstacles at a high frame rate, which has important application value in scenarios such as blind assistance or outdoor robot navigation.

1 Introduction

In recent years, rapid advances in computer vision and deep learning have led to significant progress in the areas of semantic segmentation, object detection, and multi-object tracking. These advances have opened up new avenues for real-time perception in applications ranging from autonomous driving to pedestrian assistance and service robotics. The results in these applications, especially in the field of autonomous driving, have been very prosperous. For pedestrian wearable applications, However, for pedestrian wearable device applications, accurate identification of walking areas and reliable tracking of dynamic obstacles are crucial to ensure safe navigation, especially in urban environments with dense and unpredictable traffic participants. However, many existing approaches either rely on multiple sensors (e.g., LiDAR, stereo cameras) or are primarily designed for well-structured road scenes, with gaps in purely monocular, first-person view data. Therefore, this study designs a monocular camera dataset with a first-person perspective, capturing pedestrian views in different traffic environments, which is of great relevance for walking area recognition and tracking of traffic objects in Europe.

In this context, this study proposes an integrated deep learning-based system for real-time segmentation of walking areas and detection of multi-target obstacles, focusing on monocular video streams acquired from the pedestrian perspective. Unlike automated driving with a vehicle-centric perspective, our approach is tailored specifically for monocameras worn or carried by pedestrians, which requires both adaptation to complex street environments and efficient processing to maintain high frame rates. For example, real-time segmentation of walkable areas such as sidewalks and zebra , crossings, and real-time detection and tracking of traffic participants in the current view. To meet these requirements, we collect and annotate a large self-designed first-person dataset that reflects different urban conditions such as varying light levels, pavements and bike lanes, artificial islands, and other multiple traffic situations. This dataset is the basis for training the segmentation and detection modules.

Our approach consists of two main components. Firstly, we use a state-of-the-art multi-target detection paradigm based on the YOLO family, followed by a tracking module utilising algorithms such as DeepSORT [1], ByteTrack [2] and BoT-SORT [3]. By comparing these algorithms, we quantitatively evaluate their performance in terms of detection accuracy, tracking stability and computational efficiency. Second, we also use YOLO's semantic segmentation network to delineate walking areas. This network employs data augmentation and feature refinement techniques used in training the model to reduce the impact of categories with low sample size on other categories. The experimental results show that our proposed method not only maintains high accuracy in recognising walkable areas, but also reliably detects and tracks dynamic obstacles at real-time frame rates. This capability is particularly applicable to blind assistance or outdoor robot navigation, where continuous and accurate situational awareness is crucial.

The remainder of this paper is organised as follows. In Section 2, we review related work on semantic segmentation, object detection and multi-object tracking, focusing on existing challenges in pedestrian-centric real-time applications. Section 3 describes the proposed approach in detail, including the design of the segmentation network, the selection of YOLO-based detectors, and the integration of various tracking frameworks. Section 4 describes the experimental setup, dataset collection and labelling, evaluation metrics and performance results. Finally, Section 5 concludes the paper and outlines potential future research directions in enhancing real-time pedestrian navigation solutions.

2 Related Work

2.1 Advancements in Object Detection

Object detection aims to locate and classify objects of interest in an image, which has extensive and important applications in the field of computer vision, including autonomous driving, video surveillance, pedestrian detection, etc. With the development of deep learning, object detection methods have undergone significant evolution.

Classic object detection methods are usually divided into two stages, namely, the candidate region generation stage and the target classification and positioning stage. Among them, the two-stage detection method represented by the R-CNN family (including Fast R-CNN [4] and Faster R-CNN [5])first uses the candidate region generation network (RPN) to generate candidate regions of interest, and then determines the specific location and category of the target through further classification and bounding box regression. This type of method usually relies on convolutional neural networks (CNNs) to extract rich feature information and uses a multi-stage processing flow to improve detection accuracy. However, due to its multi-step processing mechanism, the two-stage method is subject to certain limitations in inference speed and is difficult to meet application scenarios with high real-time requirements.

In contrast, single-stage target detection methods represented by YOLO (You Only Look Once) [6] and SSD (Single Shot MultiBox Detector) [7], directly implement both the position regression and category prediction of the target in the network, omitting the candidate region generation step, thus significantly improving the detection speed and real-time performance. For example, YOLO divides the image into multiple unit grids, each of which is responsible for predicting a certain number of bounding boxes and category probabilities. The simplicity and real-time processing capabilities of this method make it very popular in practical applications. SSD [7] also achieves a good balance between accuracy and speed by predicting targets using feature maps of different scales.

In recent years, the YOLO series has undergone continuous and significant optimization and development. From the initial YOLOv1 to the latest YOLO11, each iteration has been significantly improved in accuracy, speed, and the ability to detect small targets. YOLO11 further enhances its generalization ability and robustness in complex scenarios by introducing a more efficient network structure, a more effective attention mechanism, and a more flexible multi-scale feature fusion method, especially in traffic obstacle detection and mobile devices.

In addition, YOLO11 also pays special attention to the optimization of small target detection performance. Through the refined feature pyramid structure (FPN) [8] and spatial attention mechanism, the model can effectively capture the feature information of small targets in the image, thereby greatly improving the detection accuracy. Given these advantages, the YOLO series of models, especially the latest YOLO11, has become the preferred model in many real-time environmental perception tasks such as autonomous driving, pedestrian safety assistance systems, and mobile robot navigation systems.

2.2 Advancements in instance Segmentation

As one of the important tasks in the field of computer vision, instance segmentation aims to identify each object in an image and provide an accurate pixel-level mask for each instance. Compared with traditional object detection and semantic segmentation,

this task requires a clear distinction between different individuals in the same category. Therefore, it has important application value in complex tasks such as autonomous driving, robot navigation, medical image analysis, and scene understanding. Early instance segmentation methods mostly rely on traditional image processing techniques, such as contour detection and region segmentation, but they perform poorly when dealing with complex backgrounds, overlapping objects, and occlusions, and their generalization ability is insufficient. With the rise of deep learning, instance segmentation technology has made rapid progress.

Mask R-CNN proposed by He et al.[9] is an important milestone in the field of instance segmentation. It is based on the detection framework of Faster R-CNN and adds an additional mask prediction branch, which can effectively predict the fine mask of each detected target. Mask R-CNN greatly improves the accuracy of instance segmentation by jointly training object detection and mask generation, and quickly becomes the basic model for many tasks.

After Mask R-CNN, many innovative methods have been proposed. For example, the YOLACT [10] model achieves real-time and efficient instance segmentation by simultaneously predicting the prototype mask and mask coefficients. BlendMask [11] introduces a more flexible mask fusion mechanism, further improving the accuracy and speed of instance segmentation. In addition, the SOLO series models [12] proposed to generate instance masks directly from feature maps in a single-stage manner, greatly improving the real-time performance and accuracy of instance segmentation. CondInst [13] proposed a dynamic mask generation mechanism based on conditional convolution, further reducing the computational complexity and improving the inference speed and accuracy.

In recent years, the rapid evolution of the YOLO series has also promoted the progress of real-time instance segmentation technology. The latest YOLOv11 [14] network combines an efficient target detection framework with an instance mask generation mechanism. Through the optimized feature pyramid network and spatial attention mechanism, it effectively improves the segmentation performance of small targets and overlapping instances in complex scenes, especially for resource-constrained mobile devices and real-time scenes At the same time, lightweight real-time instance segmentation models have gradually gained attention, meeting the needs of mobile terminals and embedded systems for real-time processing capabilities.

2.3 Object Tracking Technology

Target tracking technology is mainly used to continuously mark and track the same target object in a video sequence, and is widely used in autonomous driving, video surveillance, behavior analysis and other fields. Early target tracking methods such as Kalman filtering and particle filtering mainly rely on target motion models. These methods have limited accuracy and are difficult to effectively handle complex occlusion, target intersection and rapid motion.

With the development of computer vision technology, multi-target tracking (MOT) technology based on deep learning has made significant progress. In recent years, the SORT algorithm [15] has attracted widespread attention for its simple structure and efficient reasoning speed. However, when SORT handles complex scenes such as target occlusion and track intersection, the tracking target is often lost. To this end, DeepSORT [1] adds deep learning appearance features (re-identification, ReID) to the SORT algorithm to enhance the accuracy of cross-frame target association, effectively improving

the robustness and accuracy of tracking.

The BoT-SORT [3] algorithm proposed in recent years further optimizes the trajectory smoothing and motion prediction mechanism, significantly enhancing the stability and continuity of tracking, and is particularly suitable for target tracking tasks in crowded scenes. In addition, the ByteTrack [2] algorithm effectively integrates high and low confidence detection results through an innovative confidence hierarchy mechanism, significantly reducing the target missed detection rate, and is particularly suitable for real-time tracking applications.

In addition, research and development in the field of multi-target tracking has also benefited from the promotion of standardized datasets and evaluation indicators. MOT datasets such as MOT15 [16], MOT16 [17], MOT19 [18], and MOT20 [19] are widely used for training and evaluation of multi-target tracking algorithms. These datasets cover a variety of complex real-world scenarios, including crowded public places, traffic intersections, streets, and indoor environments, providing clear ground truth data to support rigorous evaluation of algorithm performance. The annotation of MOT datasets usually includes information such as target bounding boxes, trajectory identification, and motion status. It has become a benchmark tool for measuring the performance of new algorithms and has greatly promoted the development and practical application of research in this field. At the same time, the performance evaluation standards for MOT datasets have gradually been standardized, usually using indicators such as MOTA (multi-target tracking accuracy), MOTP (multi-target tracking precision), IDF1 (identity consistency F1 score), HOTA (comprehensive tracking accuracy) [20], etc. These indicators provide researchers with fair and clear performance comparison standards, promoting the continuous advancement of multi-target tracking technology and the continuous improvement of actual deployment capabilities.

2.4 First-Person Perspective Application Research

First-person perspective (Ego-centric) vision has gradually gained attention in the field of computer vision research in recent years, which has triggered a large number of innovative research efforts due to its unique perspective characteristics and application scenarios.Damen et al.[21] proposed a new dataset called EPIC-KITCHENS, which provides rich first-person perspective video clips that focuses on human behaviour recognition in kitchen environments, which greatly advances research progress in behavioural understanding tasks. Similarly, Grauman et al.[22] created the Ego4D dataset, a large-scale global database of first-person perspective videos covering multiple everyday scenarios, effectively contributing to the development of visual scene understanding, social interaction analysis, and target tracking tasks. Recent studies have also focused on the application of first-person perspective in augmented reality (AR) and assisted navigation. For example, Cartillier et al. [23] developed a real-time first-person view pedestrian navigation system that utilises deep learning algorithms for pedestrian detection, environment awareness and obstacle avoidance planning, which demonstrated significant benefits in real-world navigation scenarios. Meanwhile, Kazakos et al. [24] implemented motion prediction and classification using first-person view video, which significantly improved the real-time and accuracy of virtual-reality interaction in augmented reality scenes.

3 Methodology

3.1 Overall approach flow

This study proposes an environment perception system for real-time video in first-person perspective, which is trained with YOLO model, and the dataset is a homemade constructed dataset. Its process framework is shown in Figure 1. The input video size is 1920x1080, and the video frame is reduced to 640x640 size by opencv's resize function in order to match the entrance size of the YOLO11 detection model. In order to investigate the real-time performance of different trackers and the effect of data enhancement for different parameters in the detection model, this experiment compares the performance and latency of different trackers in the same scene. Meanwhile, a better detection model is trained with modification of different data enhancement parameters.

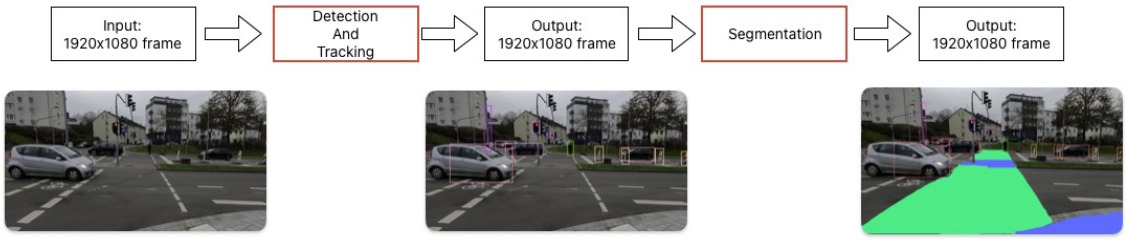

Figure 1: Schematic diagram of the overall system framework

Firstly, the video frame passes through the YOLO11 target detection module to extract traffic obstacles (e.g., vehicles, pedestrians, bicycles, etc.); subsequently, the detected target passes through the multi-target tracking module to maintain the consistency of the target identity across the frames; then the YOLO11 segmentation module identifies the walkable region; finally, all the information is superimposed on the original video frame output.

3.2 YOLO11 detection and segmentation model structure

The YOLO11 target detection network consists of three main parts: the backbone network (Backbone), the neck structure (Neck), and the detection head (Detection Head), as shown in Figure 2.

3.2.1 Model Backbone

The backbone is responsible for extracting hierarchical visual features from the input image. YOLOv11 adopts an improved lightweight design that incorporates convolutional modules such as `C3k2` and attention enhancement modules such as `C2PSA`. Early layers progressively downsample the input while increasing the number of channels to capture a rich hierarchy of features. Specifically, the following three output feature maps are extracted from the backbone, each corresponding to a different scale:

• P3 (1/8 scale): The output of layer 4, which is used to capture fine-grained spatial details for small object detection.

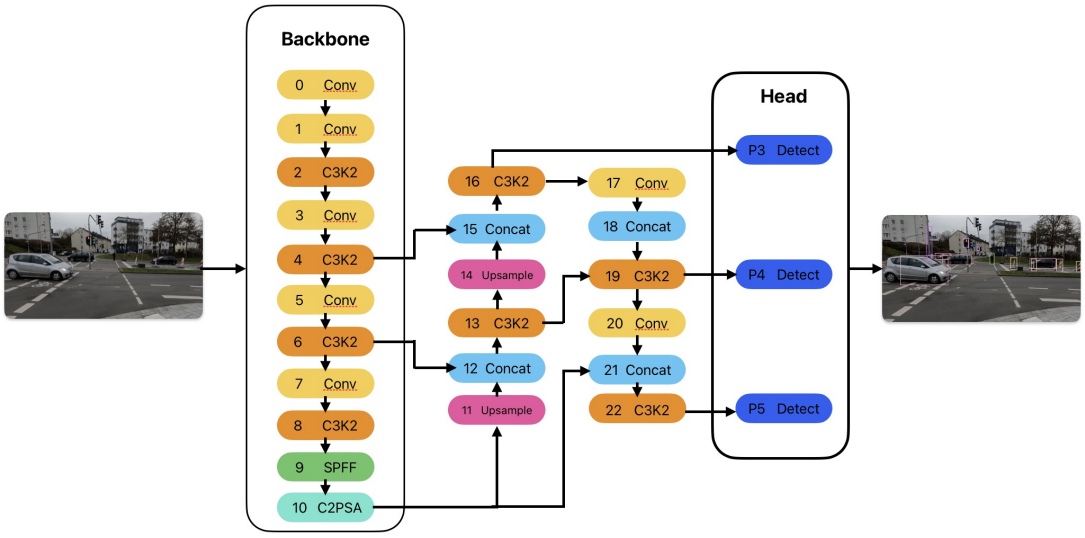

Figure 2: YOLO11n Detection Structure, Figure adapted from YOLO11 [14].

• P4 (1/16 scale): Output of layer 6, balancing semantic abstraction and resolution.
• P5 (1/32 scale): Output of layer 10, processed by a combination of spatial pyramid pooling (SPPF) [25] and `C2PSA` modules. `C2PSA` module fuses `C2f` structure with polarized self-attention (PSA) mechanism to enhance channel and spatial feature selectivity. This improves the network's ability to suppress background noise and highlight object-related regions, especially in complex scenes.

3.2.2 Model Neck

The neck structure is designed to aggregate and fuse features from different depths in the backbone network to enhance detection capabilities. YOLOv11 adopts an FPN with PANet [26] design, combining upsampling, concatenation, and an additional `C3k2` module to propagate strong semantic features from deep layers to shallow layers. The key steps of the neck include:

• Upsampling high-level features (e.g., from P5 to P4, from P4 to P3)
• Connecting with the lateral connections of the backbone network (e.g., from layer 6 and layer 4)
• Refining the fused features using multiple residual `C3k2` modules

3.2.3 Detection Head

The head contains three detection branches, corresponding to the P3, P4, and P5 feature levels. Each branch outputs bounding box coordinates, class probability, and final score.

A unique design choice of YOLOv11 is to explicitly reintroduce the deep semantic features of the last layer of the backbone network (layer 10, `C2PSA`) into the P5 detection branch. Specifically, the P4 output (layer 19) of the head is downsampled to match the resolution of P5 (1/32) and then concatenated with semantically rich features from layer 10 (through layer 21). This concatenation is followed by a refinement module to finally

generate the P5 detection output.Because in shallow features, that is, at high resolution, the model can retain more spatial details, which is important for small objects. In deep features, that is, at low resolution, more details are forgotten, and only care about which category the feature belongs to, but cannot determine the location in the original image.

This fusion enables the P5 branch to benefit from both structural features from the head pathway and high-level semantic cues extracted from the backbone network, leading to better detection of large and occluded objects.

3.2.4 Segmentation Head

YOLOv11 extends its detection architecture with a lightweight segmentation head,whose structure is shown in Figure 3. This segmentation head is the same as the backbone and neck features used for object detection, enabling efficient multi-task learning without increasing model complexity.

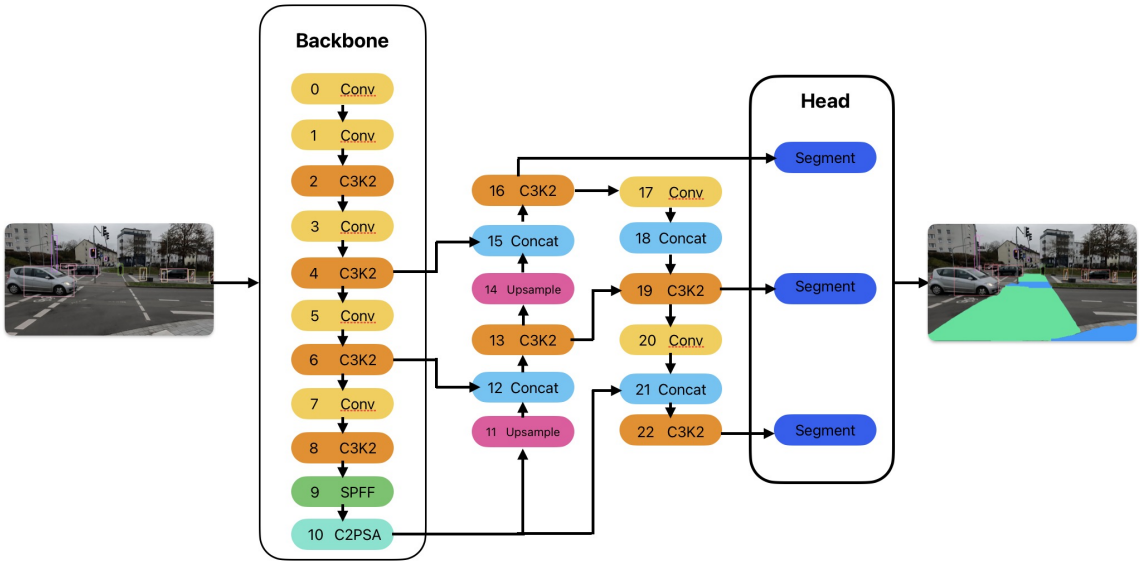

Figure 3: YOLO11n Segmentation Structure, Figure adapted from YOLO11 [14].

The segmentation module takes a fused feature map (usually from the P3 level) and generates a set of mask prototypes. For each detected object, the system predicts a set of mask coefficients and its bounding box and class label in parallel. The final instance mask is then calculated by linearly combining these prototypes with the corresponding coefficients. Finally, a mask and bbox are output for each object.

YOLOv11-seg implements the instance segmentation task, which extracts independent regions for each object by simultaneously predicting the object detection box (position + category) and the object mask (polygon/pixel level). This design, inspired by YOLACT [10] and YOLOv8-seg [27], is both memory-efficient and fast, making it ideal for real-time applications. The segmentation head is jointly trained with the detection head, allowing the model to simultaneously predict the object location and its corresponding segmentation mask.

3.3 Multi-target tracking algorithm structure

In the object tracking stage, three methods, BoT-SORT, ByteTrack, and DeepSORT, were used in this study for comparative analysis respectively:

3.3.1 ByteTrack structure

After being proposed by Zhang et al. [2], ByteTrack has achieved quite outstanding results. Its feature is that it can take different processing methods according to high-confidence detection frames and low-confidence detection frames. As in Figure 4 shown, when processing low-confidence detection frames, they may be reconfirmed as tracking states in the future. Instead of choosing to delete them like the traditional MOT algorithm. Especially when tracking the occlusion problem of objects that are of particular concern, the target will cause the detector's Conf to drop due to occlusion. At this time, the tracked target is matched with the current low-Conf target. This match is based on the intersection of the predicted box and the detection bounding box. When the match is successful, the Kalman update covariance, that is, the trajectory matrix. Even if the match is not successful, it will be changed to the lost state and continue to participate in the next match. Within the max time age number, if the first match has not been completed, this trajectory will be deleted. At this time, it means that the object is already outside the video range, or the occlusion time is too long, exceeding the set time range. In general, the ByteTrack is very dependent on the accuracy of the detector, but compared with other trackers, it is very good in real time.

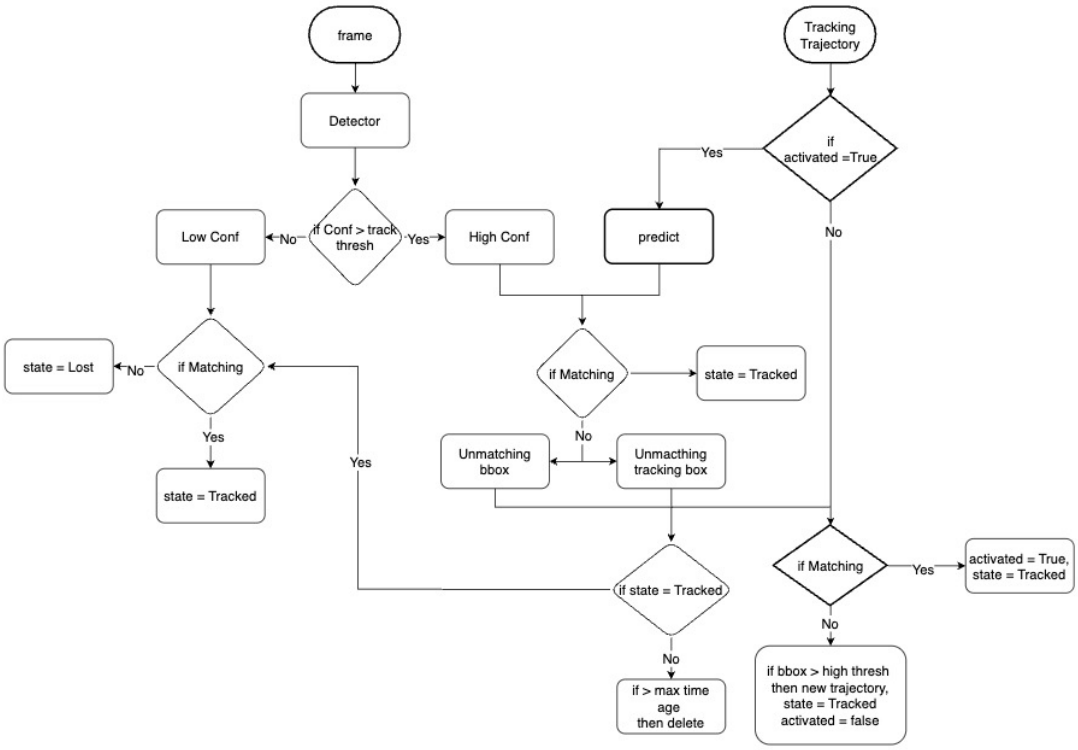

Figure 4: ByteTrack workflow structure, Figure adapted from ByteTrack [2].

3.3.2 BoT-SORT structure

BoT-SORT is a modern tracker that improves on the classic SORT algorithm by integrating a lightweight ReID feature extractor (e.g., OSNet) and a two-stage association strategy of motion cmcs and an exponential moving average mechanism to stabilize trajectory updates.

As in Figure 5 shown, for each input video frame, a pre-trained detector (in this case, YOLOv11) first generates a set of object bounding boxes with associated class labels and confidence scores. All existing object trajectories maintained by the tracker are updated using a Kalman filter to predict their current position based on previous motion. In the first stage of matching, high-confidence boxes are used to perform IoU and ReID joint matching with predicted trajectories to improve identity recognition accuracy. For unmatched trajectories, low-confidence boxes are used for the second stage of pure IoU matching to improve recall. Each object is represented by a 512-dimensional embedding vector. Matched trajectories are updated with the current detection box and the embedding vector. Unmatched high-confidence detections are initialized as new tracks for a second match. If the inactivity duration of an unmatched track exceeds a threshold, it is marked as terminated. To reduce short-term jitter, in track management, BoT-SORT applies an exponential moving average to update the center position of each track, thereby improving visual smoothness and ID stability across frames.

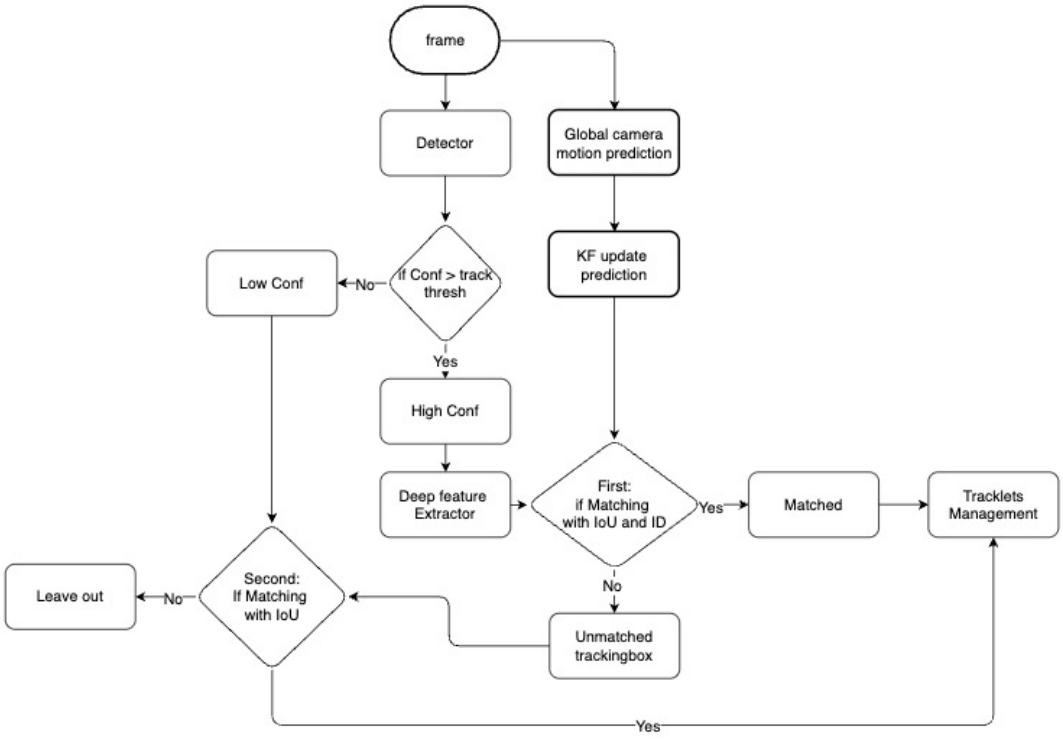

Figure 5: BotSORT workflow structure, Figure adapted from BoT-SORT [3].

3.3.3 DeepSORT structure

As shown in the figure 6, the deepsort process is based on the SORT algorithm with the addition of Cascade matching and ReID models. After the detector obtains the bounding box, category and confidence of the target, each area is extracted into a feature vector of a certain length through the pre-trained ReID model. At the same time, for the existing trajectory, the Kalman filter is used to predict the position, and after obtaining the estimated state, the first matching is entered. In the cascade matching, a cost matrix is constructed between the detection result and the predicted trajectory. The matrix consists of the distance between the predicted box by the Kalman filter and the detection box detected by detector, and the distance between the appearance features. Finally, different weights are assigned to form the cost matrix. The second matching is based on IoU, just like the SORT algorithm. The successfully matched trajectory uses the current detection box to update the state and update its appearance features at the same time. The unmatched high-confidence detection box can be initialized as a new trajectory. As the number of unmatched trajectories increases, they will be deleted until they exceed the max time age. We track the detection output frame of YOLO11 as

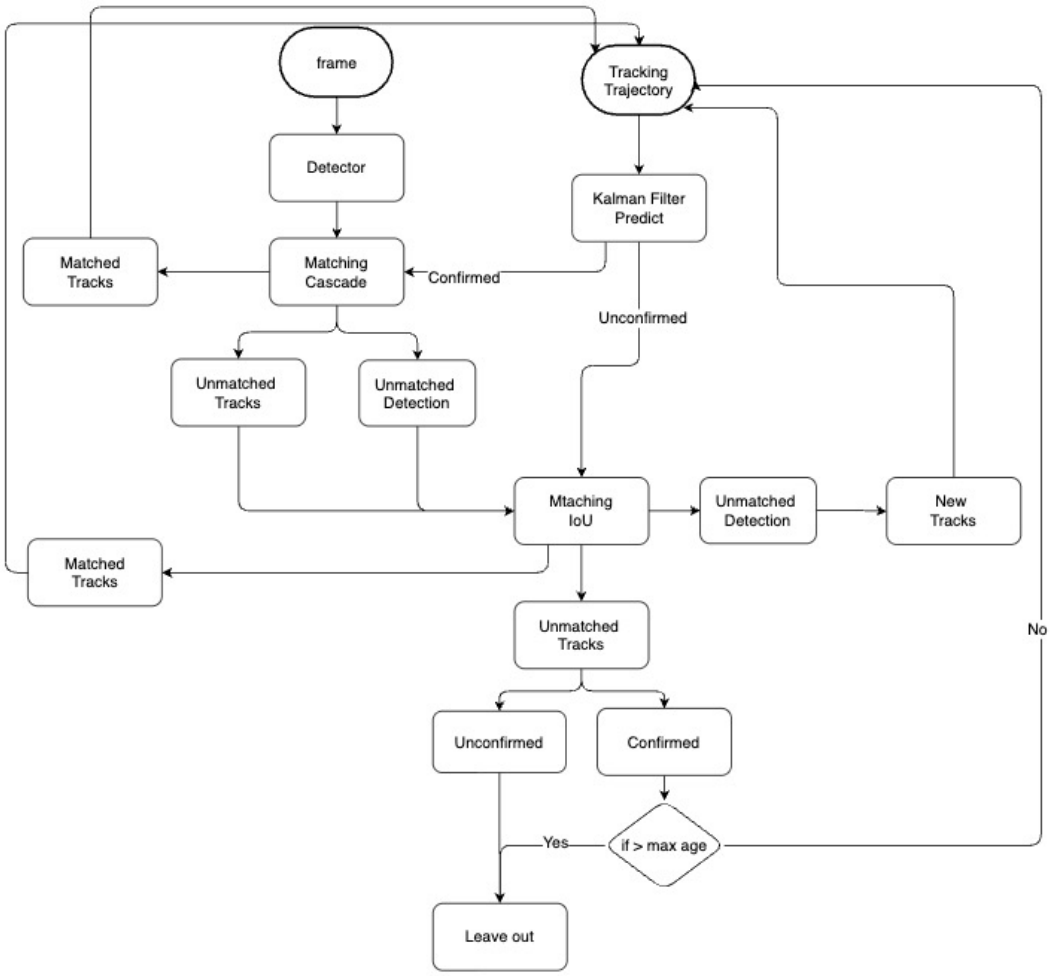

Figure 6: DeepSORT workflow structure, Figure adapted from DeepSORT [1].

the input of BoT-SORT, ByteTrack, and DeepSORT, and evaluate the performance of the three methods fairly under the same hardware and environment. The key parameter

configurations (e.g., IOU threshold, confidence threshold, ReID model selection) maintain the standard recommended values for each tracking algorithm to guarantee the objectivity of the evaluation.

4 Experiment

4.1 Experiment introduction

This experiment is divided into 3 main parts, the production of the dataset, comparing the real-time performance of the 3 trackers and comparing the presentation of the results of using the data enhancement algorithm in the training of the YOLO model. The experimental environment of this study was set up on the NVIDIA DGX computing platform, and the PyTorch framework was used to implement the training and testing of the algorithms. The dataset is a self-manufactured dataset, produced by the Department of Mechanical and Process Engineering[1] University of Kaiserslautern-Landau in German.

4.2 Dataset introduction

The VASCO dataset is captured by a monocular camera and focuses on urban traffic scenes in first-person view, covering a wide range of obstacle types such as pedestrians, vehicles, and bicycles. The whole dataset is manually annotated online in Label Studio platform [28]. The initial data is a video captured by a monocular camera with a single person carrying the camera on his chest and completing the acquisition of a typical European German city street in a walking position. The dataset continues to be updated and has a cumulative total of 150,000 frames. Valid scenes are selected for their labelling. When labelling the data, it is divided into pedestrian actions, area segmentation, object target, object status and object locations. This dataset provides both current pedestrian information and real-time environmental information and its causal relationship. At the meantime it provides a good correlation for subsequent behavioural analysis and prediction.The categories of the dataset are shown in Table 1, with a total of 28 traffic target recognition objects and 7 as segmentation regions. There are also 6 as the current action taken by the pedestrian and 5 as the action and state that the target object is in..

4.3 data enhancement algorithm

Next, I trained the detection model and the segmentation model separately. I used about 3000 labelled images, divided into training and validation sets in a ratio of 8:2. The distribution of training categories for the entire dataset inklusive about detection and segmentation is shown in Fig 7.

In the detection model, I first compared the difference under batch=32 and batch=16. The model training results are all very close as can also be seen in Figure 8. In the segmentation model, I similarly did the same comparison and the results as found in Fig 9 are also very close. Given that the model is based on the yolo11n model and the training set is small, I chose to train the model with batch=16 to ensure that the model can be generalised.

[1]petrit.rama@mv.uni-kl.de; naim.bajcinca@mv.uni-kl.de

Table 1: Classes of the dataset

Action	**Area**	**Traffic Object Name**	**Obstacle Status**	**Obstacle Location**
stop	zebra	red_light_p	walking	front
straight	sidewalk	green_light_p	runing	right
right	crosswalk	orange_light_p	standing	left
right-right	traffic_island	red_light_v	parked	near
left	pedestrian_crossing	green_light_v	moving	away
left-left	stairs	orange_light_v		
	bike_lane	pedestrian_crossing_sign		
		pedestrian_path_sign		
		pedestrian_prohibited_sign		
		pedestrians_must_cross_road_right_sign		
		pedestrians_must_cross_road_left_sign		
		traffic_calming_zone_sign		
		bus_stop_sign		
		danger_children_sign		
		warning_sign		
		person		
		car		
		bus		
		van		
		truck		
		motorcycle		
		bicycle		
		scooter		
		trash_bin		
		pole		
		tree		
		handrail		
		construction_barriers		

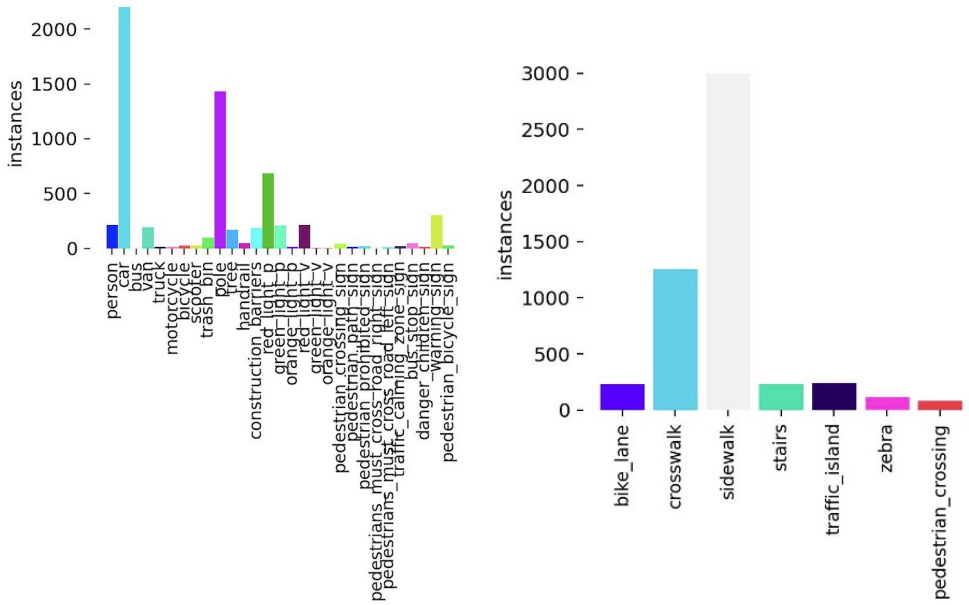

Figure 7: Training data categories and quantity:Detection(Left) Segmentation(Right)

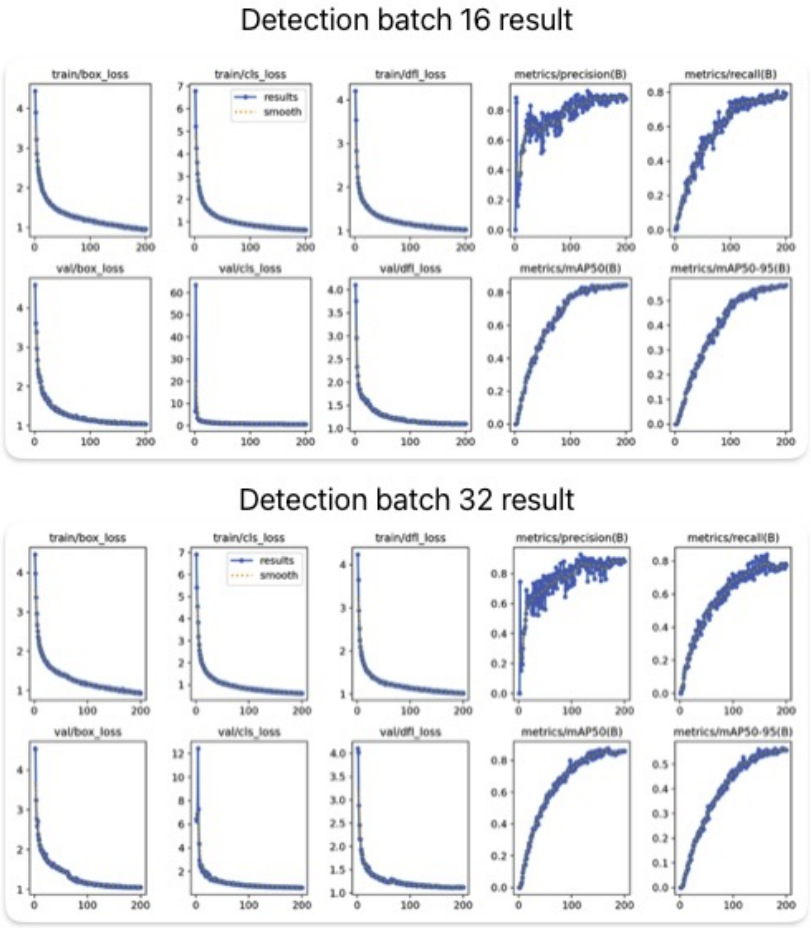

Figure 8: Detection training results of batch=16(top) and batch=32(under)

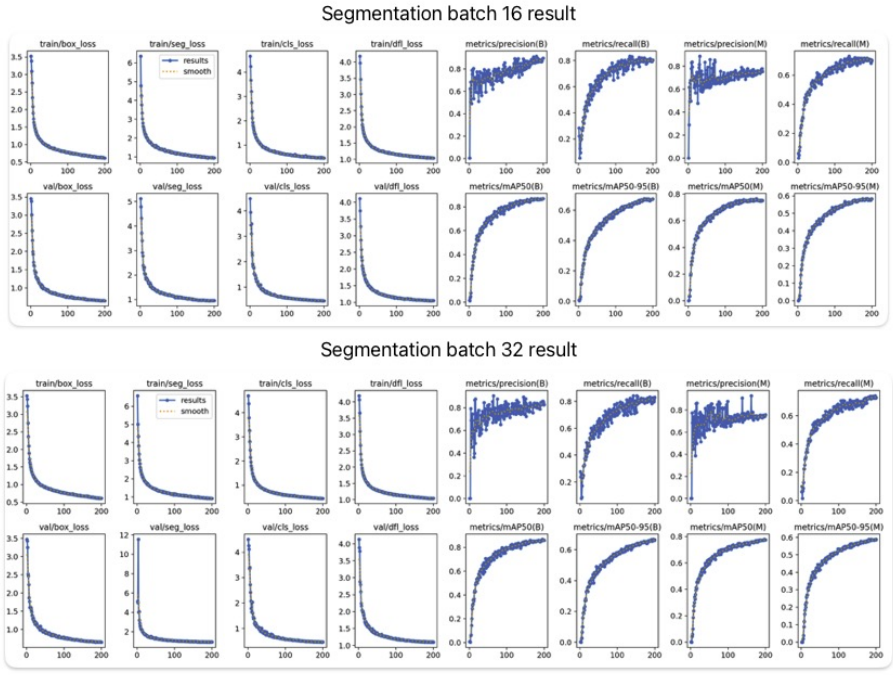

Figure 9: Segmentation training results of batch=16(top) and batch=32(under)

I then started the comparison of image enhancement algorithms. I kept augment true, adjusted degree to 10, scale to 1, mixup to 0.3 and copy paste to 0.3. The results of the model training are shown in Figure 10. As shown in the figure, the F1 scores and prediction regression curves After adding image enhancement, the performance decreases instead. F1 decreases from 0.78 to 0.52 Under the same confidence condition. The average prediction of all classes decreases from 0.845 to 0.604 . Considering the uneven distribution of the data, it is possible that the enhanced images generate low-quality positive samples or overfitted pseudo-negative samples. For example, increasing the degree distorts the edges of the image and affects the calculation of IoU. For example, mixup strategy will make the image mixed, which may 'pollute' the data distribution when the scene is complex or the background is inconsistent, causing the model to learn confusing features. In the case of small targets, the features learnt by the detector will be unclear.

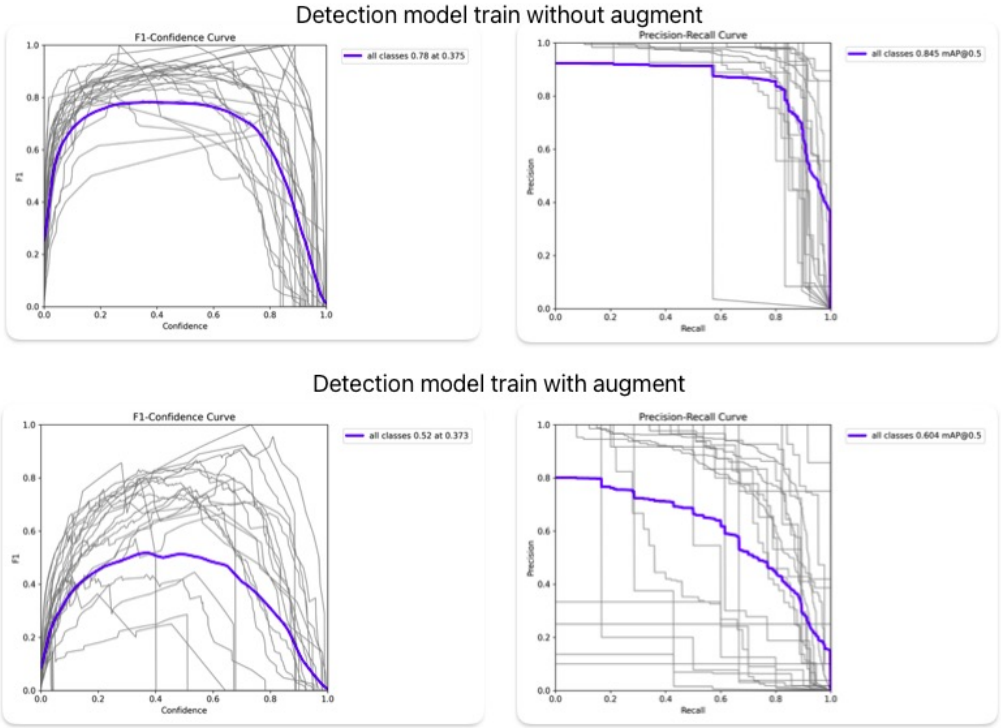

Figure 10: Classes=29 without augment(top) ; Classes=29 with augment(under)

Considering that some of the categories only exist in certain scenarios, and that the distribution of the number of targets in the data is extremely unbalanced,for some classes is the number under 10 times.So I removed a few categories with small amounts of data. Continuing to train the model with the same data enhancement parameters, the results are shown in Figure 11. After removing some extremely sparse number of categories, the performance of the model effectively improves on F1 and predictive regression. The average optimal F1 from 0.52 rises to 0.76, and the optimal IoU also rises from 0.373 to 0.522, and the average prediction of the whole category under map@0.5 also rises from 0.604 to 0.826. This indicates that the data-enhanced correlation strategy can indeed improve the correlation performance of the model after removing the extremely small data categories.

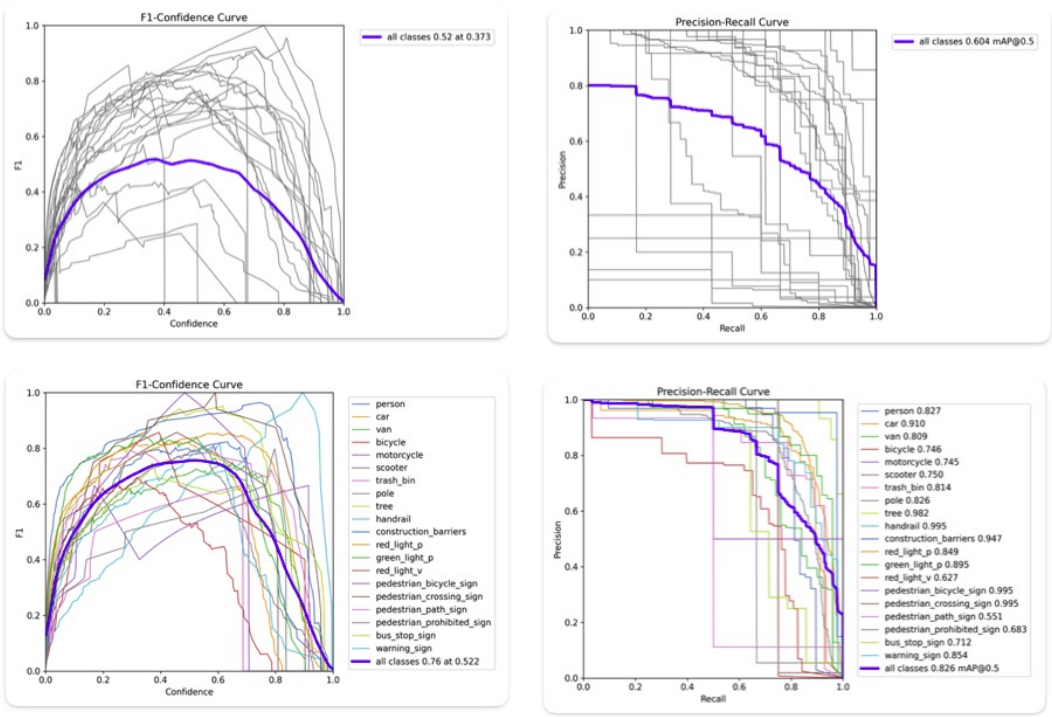

Figure 11: Classes=29 with augment(top) ; Classes=20 with augment(under)

4.4 comparing the real-time performance of the 3 trackers

When comparing the performance of the three trackers, I mainly did two parts of work. One was to use the CVAT annotation platform [29] to annotate a video data frame by frame in the Vasco dataset. The data belonged to a 1080p and 23fps first-person pedestrian traffic video, with a total of 2209 frames. After all the annotations were completed, they were exported to the MOT format. This document is mainly used as ground truth to detect the performance of the tracker. The document mainly focuses on traffic categories such as cars, poles, traffic lights, and pedestrians. In order to avoid errors caused by too few number of categories, I chose person as the evaluation category. The evaluation tool uses the open source tool TrackEval [30]. And HOTA [20], CLEAR [31], and Identity [32] (IDF1)are selected as evaluation indicators. HOTA mainly evaluates the overall accuracy, where Deta is the accuracy of the detection part, and ASSA is the scale of ID retention ability. Clear is a classic comprehensive indicator created by Bernardin et al., including FN, FP, etc. IDF1 reflects the proportion of TP in the whole, and the higher the score, the better.

At the same time, in order to make the experiment more comparable, I replaced the Embedder interface of BotSORT and DeepSORT with OSNet [33]. Since OSNet is a model designed and trained specifically for pedestrian re-identification (ReID), its feature learning targets: clothing color, human body contour, local body texture. So in theory, this model is very helpful when tracking the human body. In the middle of the gt(groundtruth) video, there is a part where pedestrians are waiting for the red light at the intersection. Under the occlusion of the car, whether the pedestrian's ID flashes or changes is a test of the tracker's performance. Therefore, the difficulty of this part is to disassemble the backbone of the OSNetx0.25 model and merge it with the pipeline of the tracker.

The second part is an experiment on the real-time performance of the three trackers. All trackers use YOLO11 as the detector, and all add YOLO11 segmentation. Inference is performed on the NVIDIA DGX server station. The results are based on the entire video processing time, in seconds. The entire test code process includes traffic target detection and tracking, and traffic area segmentation. We found 5 videos for experiments, including different scenes and different weather conditions, and finally calculated the average processing time.

5 Results and Discussion

5.1 Summary of Results

This study analyzed the effectiveness of three different tracking algorithms (BoT-SORT, ByteTrack, DeepSORT), batch configuration, and various data augmentation techniques in YOLO model training through systematic experiments. The experiments were based on the self-developed VASCO first-person pedestrian traffic dataset and the NVIDIA DGX computing station. Pedestrians can achieve traffic target recognition and tracking, as well as the division of walkable areas through this study, laying the foundation for the subsequent development of walking assistance systems.

5.2 Effect of Batch Size

Batch size comparison during YOLO model training shows that a larger batch size (batch size of 32) slightly improves mAP and IoU performance metrics compared to a smaller batch size (batch size of 16).As shown in the Figure 12 and Figure 13 However, a larger batch size slightly reduces processing speed. Considering the balance between generalization ability and computational efficiency, using a batch size of 16 is considered the best choice in this research setting, especially when the dataset is relatively small and computational resources are limited.

5.3 Data Augmentation Techniques

Preliminary experiments combined data augmentation methods such as data augmentation, dimensionality adjustment, mixup, and copy-paste, but the results showed unexpected performance degradation. Specifically, the F1 score drops significantly from 0.78 to 0.52, and the Precision-Recall performance degrades, suggesting that some augmentation strategies may negatively impact detection accuracy by introducing distorted or inconsistent visual cues. The observed degradation in performance likely arises because aggressive augmentation methods, such as mixup and copy-paste, inadvertently introduce unrealistic object combinations and perspectives that do not naturally occur in real-world first-person scenarios, thereby hindering model generalization.

Further analysis shows that categories with extremely limited data samples are most significantly affected by data augmentation. Removing these sparse categories from the training set significantly improves performance metrics. After category adjustment, the F1 score increases from 0.52 to 0.76, the IoU increases from 0.373 to 0.522, and the mAP@0.5 significantly improves from 0.604 to 0.826. As shown in Table 2. This improvement clearly illustrates the critical balance required between data quantity and quality. It also suggests that careful data curation might have a greater impact on model

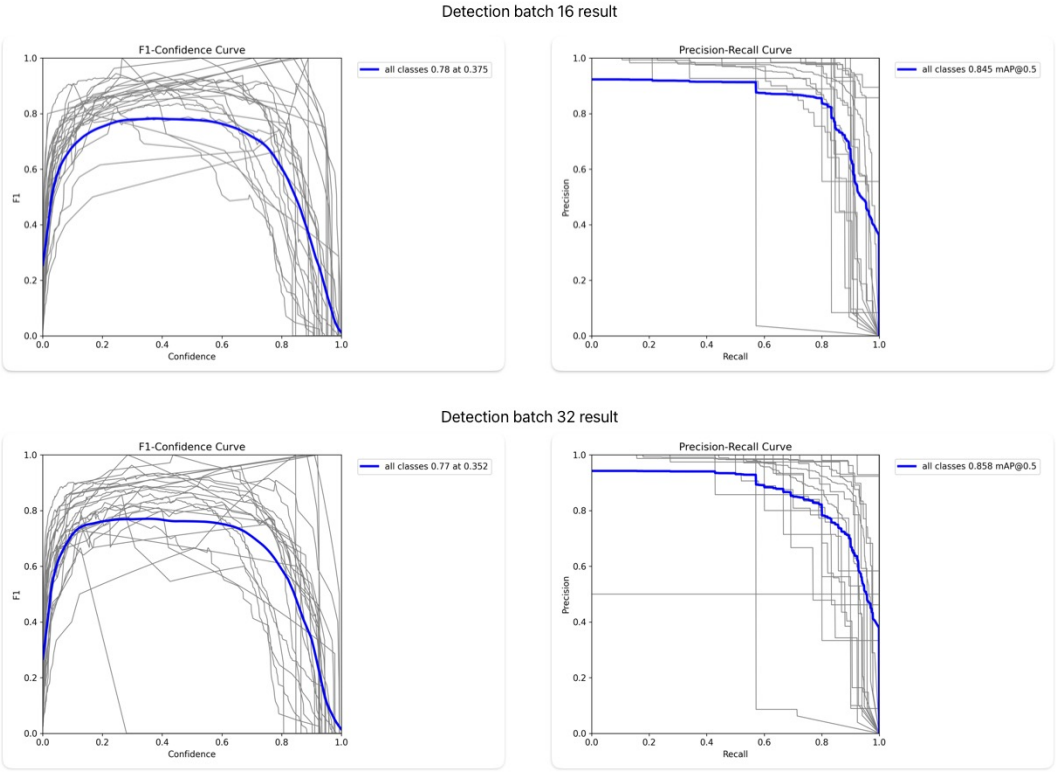

Figure 12: F1-Confidence Curve and Precision-Recall Curve.

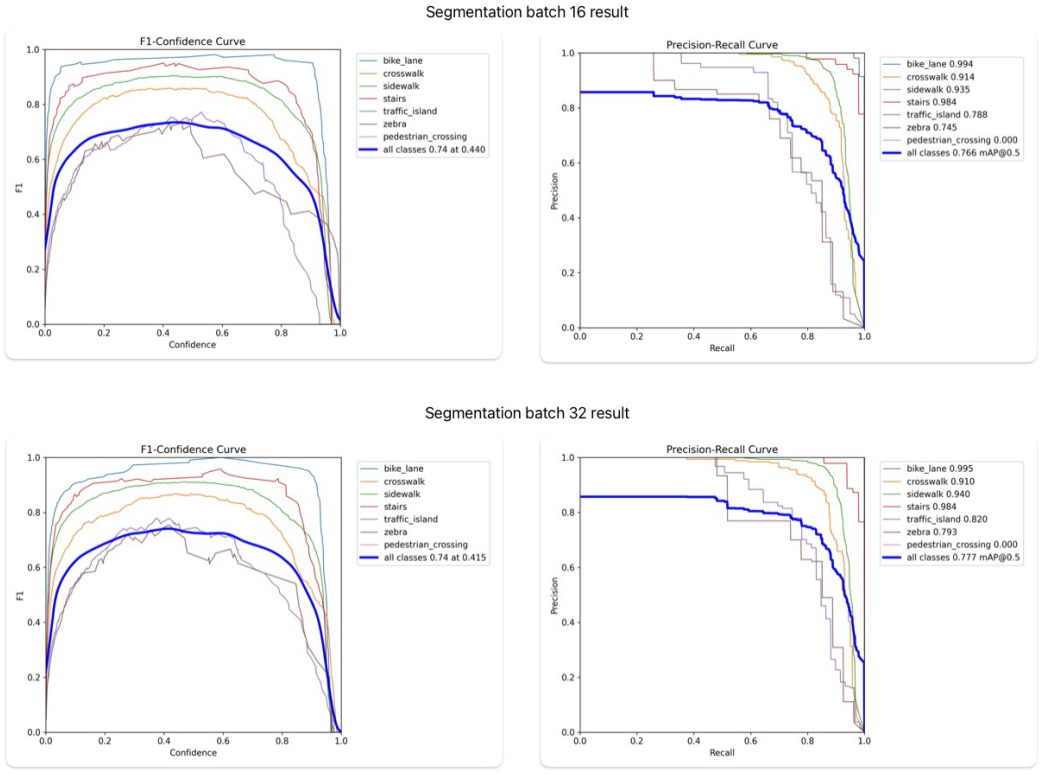

Figure 13: F1-Confidence Curve and Precision-Recall Curve.

performance than aggressive augmentation, particularly in contexts of limited data availability.

Table 2: Comparison of augmentation and class selection strategies.

Training setting	**F1** ↑	**IoU** ↑	**mAP@0.5** ↑
Classes=29, no augmentation	0.78	0.375	0.845
Classes=29, with augmentation	0.52	0.373	0.604
Classes=20, with augmentation	0.76	0.522	0.826

5.4 Tracker Performance Analysis

By comparing and analyzing the three trackers BoT-SORT, ByteTrack, and DeepSORT, we can gain an in-depth understanding of their real-time processing capabilities and tracking accuracy. The detailed quantitative evaluation results are summarized, which clearly show the advantages and disadvantages of each tracker in the pedestrian tracking task using the VASCO dataset.

Table 3: 3 Tracker Performance Compare

Tracker	**HOTA** ↑	**DetA** ↑	**AssA** ↑	**IDF1** ↑	**MOTA** ↑	**FP** ↓	**FN** ↓
BoT–SORT(OSNet)	30.31	37.34	24.63	47.87	38.07	246	310
ByteTrack	**40.81**	36.98	**45.04**	**67.35**	36.07	269	**307**
DeepSORT(OSNet)	36.35	**38.17**	34.62	59.06	**38.29**	**235**	330

Based on the evaluation indicators (HOTA, CLEAR, IDF1) of TrackEval [30] based on real data in MOT format in Table 3 and the processing time of 5 test videos (resolution of 1080, 23fps) in Table 4, I came up with the following analysis. BoT-SORT achieves effective target association. The principle is that after calling OSNet for Reid for each frame, it enters GMC to complete global compensation and two-step rematching, but the overall processing speed is reduced due to the processing operation of each frame. And the detection and association scores are relatively low. ByteTrack has excellent overall performance, and obtained the highest scores in both HOTA and IDF1 indicators, indicating that it has achieved a good balance in detection accuracy and association accuracy. Since it can effectively utilize high-confidence and low-confidence detection, does not extract appearance features, and completely relies on IOU and two-stage matching, the time spent on processing each frame only comes from the reasoning of YOLO11, and it is very good in real-time performance. In contrast, DeepSORT is in the middle in terms of HOTA score, but has excellent results in FP score, indicating the least false detection. Compared with ByteTrack, it sacrifices about 14% of real-time performance in exchange for more stable ID tracking and lower false detection.

The comparison results highlight the key trade-offs between accuracy, robustness, and computational efficiency of the three trackers. ByteTrack is the most balanced choice in real-time scenarios. It provides strong recognition capabilities and efficient computing performance. It is the best choice when the real-time requirement is greater than the false

Table 4: Running Time of each tracker on 5 test videos

Tracker	Video	Frames	Time (s)	FPS	Avg. FPS
BoT-SORT(OSNet)	GX010024	1271	126.77	10.03	**8.18**
	GX010026	2212	234.02	9.45	
	GX010058	3226	418.53	7.71	
	GX010061	3764	527.63	7.13	
	GX010067	2208	334.98	6.59	
ByteTrack	GX010024	1271	85.62	14.84	**12.26**
	GX010026	2212	164.00	13.49	
	GX010058	3226	263.20	12.26	
	GX010061	3764	399.67	9.42	
	GX010067	2208	195.11	11.32	
DeepSORT(OSNet)	GX010024	1271	94.53	13.45	**10.59**
	GX010026	2212	172.57	12.82	
	GX010058	3226	339.32	9.51	
	GX010061	3764	393.06	9.58	
	GX010067	2208	289.54	7.62	

detection requirement. When stable ID tracking is required, deepsort may be the best choice. Given that the matching method in BoT-SORT is not the traditional Hungarian matching but the global GMC matching, in theory, BoT-SORT may have more advantages in terms of motion.

6 Conclusion and Future Work

6.1 Conclusion

In this study, we develop and evaluate a comprehensive real-time traffic perception system designed for first-person urban navigation scenarios. The system integrates the YOLO11 detection and segmentation model and three advanced tracking algorithms - BoT-SORT, ByteTrack and DeepSORT - to accurately detect, track and segment various urban traffic obstacles and walkable areas.

Experiments are conducted on the self-developed VASCO dataset, which contains rich European urban scenes and carefully annotated data, which makes up for the lack of outdoor first-person datasets to a certain extent. Our results highlight the effectiveness of YOLO11's powerful detection and segmentation capabilities after training on a properly balanced and fully augmented dataset. By building a self-made MOT dataset and using the TrackEval tool to compare HOTA, CLEAR, IDF1 and other indicators, it is shown that the ByteTrack tracker algorithm is more suitable for scenarios requiring real-time performance.

The exploration of data augmentation techniques highlights the key impact of dataset composition and augmentation strategy on model performance. Initially, aggressive aug-

mentation methods will degrade performance due to the introduction of inconsistent and distorted visual data. Adjustment by removing underrepresented classes significantly improves model accuracy, highlighting the importance of maintaining a balanced data distribution.

6.2 Future work

Future research should continue to improve real-time performance while improving detection and segmentation accuracy. Possible directions include:

Architecture inprove: In the above research process, the detection, tracking, and segmentation processes proceed in a linear order, but in the model architecture of YOLO11, detection and segmentation share the same backbone. Therefore, the above research actually runs the model twice when processing one frame, affecting real-time performance. Therefore, we can try to introduce a new model architecture so that the output is a mask for segmentation and detection. At the same time, we can also insert the module of the attention module into the model architecture to improve the model accuracy and efficiency.

Extended dataset development: Expand the VASCO dataset to cover a wider range of scenarios and challenging conditions, such as nighttime, bad weather, and more diverse pedestrian behaviors. Focus on collecting categories with a small number of samples to further improve the robustness and generalization of the system. For the feasibility of subsequent planning, possible walking trajectories can also be continued to be annotated on the dataset.

Tracker optimization: Explore hybrid tracker strategies, combining the advantages of different tracking algorithms or integrating motion prediction models to enhance robustness and tracking consistency under challenging conditions. It is also possible to train a multi-target tracking ReID model based on this dataset to make tracking more stable.

Application development: Based on the perception results, continue to integrate the decision information and spatial information on the data set to achieve end-to-end output under a pure visual solution. For example, deploy this system on a wearable device, input real-time video and output reliable and safe walking trajectories.

In summary, the continued development and optimization of these components will improve the practicality and reliability of the first-person perspective traffic perception system, thereby significantly promoting safer and more efficient urban travel solutions.

References

[1] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," Mar. 2017. arXiv:1703.07402 [cs].

[2] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object Tracking by Associating Every Detection Box," in *Computer Vision – ECCV 2022* (S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, eds.), (Cham), pp. 1–21, Springer Nature Switzerland, 2022.

[3] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust Associations Multi-Pedestrian Tracking," July 2022. arXiv:2206.14651 [cs].

[4] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2015.

[5] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016.

[7] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in *Computer Vision – ECCV 2016* (B. Leibe, J. Matas, N. Sebe, and M. Welling, eds.), (Cham), pp. 21–37, Springer International Publishing, 2016.

[8] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

[9] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[10] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "Yolact: Real-time instance segmentation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

[11] H. Chen, K. Sun, Z. Tian, C. Shen, Y. Huang, and Y. Yan, "Blendmask: Top-down meets bottom-up for instance segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[12] X. Wang, R. Zhang, C. Shen, T. Kong, and L. Li, "Solo: A simple framework for instance segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 8587–8601, 2022.

[13] Z. Tian, C. Shen, and H. Chen, "Conditional convolutions for instance segmentation," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I 16*, pp. 282–298, Springer, 2020.

[14] G. Jocher and J. Qiu, "Ultralytics yolo11," 2024.

[15] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*, pp. 3464–3468, 2016.

[16] L. Leal-Taixé, A. Milan, I. Reid, S. Roth, and K. Schindler, "MOTChallenge 2015: Towards a benchmark for multi-target tracking," *arXiv:1504.01942 [cs]*, Apr. 2015. arXiv: 1504.01942.

[17] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," *arXiv:1603.00831 [cs]*, Mar. 2016. arXiv: 1603.00831.

[18] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé, "CVPR19 tracking and detection challenge: How crowded can it get?," *arXiv:1906.04567 [cs]*, June 2019. arXiv: 1906.04567.

[19] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé, "Mot20: A benchmark for multi object tracking in crowded scenes," *arXiv:2003.09003[cs]*, Mar. 2020. arXiv: 2003.09003.

[20] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "Hota: A higher order metric for evaluating multi-object tracking," *International Journal of Computer Vision*, pp. 1–31, 2020.

[21] D. Damen, H. Doughty, G. M. Farinella, A. Furnari, E. Kazakos, J. Ma, D. Moltisanti, J. Munro, T. Perrett, W. Price, and M. Wray, "Rescaling Egocentric Vision: Collection, Pipeline and Challenges for EPIC-KITCHENS-100," *Int J Comput Vis*, vol. 130, pp. 33–55, Jan. 2022.

[22] K. Grauman, A. Westbury, E. Byrne, Z. Chavis, A. Furnari, R. Girdhar, J. Hamburger, H. Jiang, M. Liu, X. Liu, M. Martin, T. Nagarajan, I. Radosavovic, S. K. Ramakrishnan, F. Ryan, J. Sharma, M. Wray, M. Xu, E. Z. Xu, C. Zhao, S. Bansal, D. Batra, V. Cartillier, S. Crane, T. Do, M. Doulaty, A. Erapalli, C. Feichtenhofer, A. Fragomeni, Q. Fu, A. Gebreselasie, C. Gonzalez, J. Hillis, X. Huang, Y. Huang, W. Jia, W. Khoo, J. Kolar, S. Kottur, A. Kumar, F. Landini, C. Li, Y. Li, Z. Li, K. Mangalam, R. Modhugu, J. Munro, T. Murrell, T. Nishiyasu, W. Price, P. R. Puentes, M. Ramazanova, L. Sari, K. Somasundaram, A. Southerland, Y. Sugano, R. Tao, M. Vo, Y. Wang, X. Wu, T. Yagi, Z. Zhao, Y. Zhu, P. Arbelaez, D. Crandall, D. Damen, G. M. Farinella, C. Fuegen, B. Ghanem, V. K. Ithapu, C. V. Jawahar, H. Joo, K. Kitani, H. Li, R. Newcombe, A. Oliva, H. S. Park, J. M. Rehg, Y. Sato, J. Shi, M. Z. Shou, A. Torralba, L. Torresani, M. Yan, and J. Malik, "Ego4d: Around the world in 3,000 hours of egocentric video," 2022.

[23] V. Cartillier, Z. Ren, N. Jain, S. Lee, I. Essa, and D. Batra, "Semantic MapNet: Building Allocentric Semantic Maps and Representations from Egocentric Views," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 964–972, May 2021. Number: 2.

[24] E. Kazakos, J. Huh, A. Nagrani, A. Zisserman, and D. Damen, "With a little help from my temporal context: Multimodal egocentric action recognition," 2021.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 9, pp. 1904–1916, 2015.

[26] K. Wang, J. H. Liew, Y. Zou, D. Zhou, and J. Feng, "Panet: Few-shot image semantic segmentation with prototype alignment," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

[27] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," 2023.

[28] M. Tkachenko, M. Malyuk, A. Holmanyuk, and N. Liubimov, "Label Studio: Data labeling software," 2020-2025. Open source software available from https://github.com/HumanSignal/label-studio.

[29] CVAT.ai Corporation, "Computer Vision Annotation Tool (CVAT)," Nov. 2023.

[30] A. H. Jonathon Luiten, "Trackeval." https://github.com/JonathonLuiten/TrackEval, 2020.

[31] K. Bernardin and R. Stiefelhagen, "Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics," *EURASIP journal on image and video processing*, vol. 2, p. Art.Nr.: 246309, 2008. Publisher: Hindawi.

[32] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, "Performance Measures and a Data Set for Multi-target, Multi-camera Tracking," in *Computer Vision – ECCV 2016 Workshops* (G. Hua and H. Jégou, eds.), (Cham), pp. 17–35, Springer International Publishing, 2016.

[33] K. Zhou, Y. Yang, A. Cavallaro, and T. Xiang, "Omni-scale feature learning for person re-identification," 2019.